10.4 @ElementCollection

Although it is not an entity association type, the `@ElementCollection` is very similar to the unidirectional `@OneToMany` relationship. To represent collections of basic types (e.g. `String`, `int`, `BigDecimal`) or embeddable types, the `@ElementCollection` must be used instead. If the previous associations involved multiple entities, this time, there is only a single `Post` entity with a collection of `String` comments.

Value types inherit the persistent state from their parent entities, so their lifecycle is also bound to the owner entity. Any operation against the entity collection is going to be automatically materialized into a DML statement.

From a database perspective, there is one child table holding both the foreign key column and the collection element value.

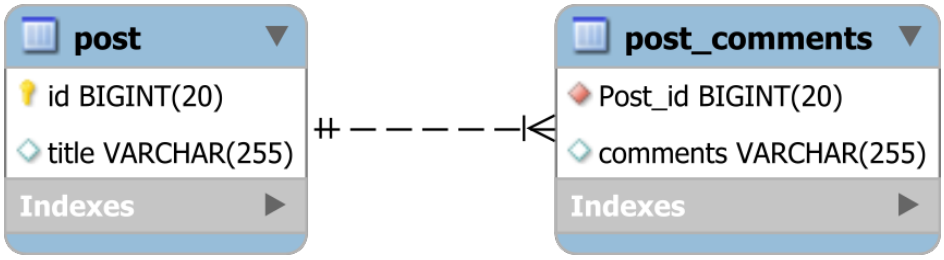

Figure 10.7: The @ElementCollection table relationship

10.4.1 @ElementCollection List

Let's consider the use case when a `List` is used with `@ElementCollection`:

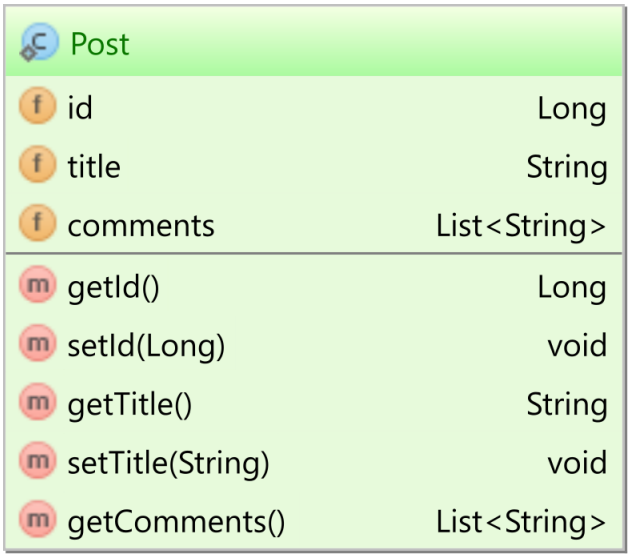

Figure 10.8: The @ElementCollection relationship

The mapping for the `comments` collection looks as follows:

```
@ElementCollection
private List<String> comments = new ArrayList<>();
```

When it comes to adding or removing child records, the `@ElementCollection` behaves like a unidirectional `@OneToMany` relationship, annotated with `CascadeType.ALL` and `orphanRemoval`.

To persist three comments, the data access layer only has to add them to the parent entity collection:

```
post.getComments().add("My first review");
post.getComments().add("My second review");
post.getComments().add("My third review");
```

Hibernate issues the insert statements during Persistence Context flushing:

```
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My first review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My second review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My third review')
```

Unfortunately, the remove operation uses the same logic as the unidirectional `@OneToMany` association, so when removing the first collection element:

```
post.getComments().remove(0);
```

Hibernate deletes all the associated child-side records and re-inserts the in-memory ones back into the database table:

```
DELETE FROM Post_comments WHERE Post_id = 1
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My second review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My third review')
```

Using an `@ElementCollection` with a `List` is not very efficient because the association is considered to be a bag, in Hibernate terminology.

A bag does not guarantee that elements are uniquely identifiable, hence Hibernate needs to delete and reinsert the elements associated with a given parent entity whenever a change occurs to the `@ElementCollection` `List`.

10.4.2 @ElementCollection Set

On the other hand, when using a `Set`, the `@ElementCollection` no longer behaves like a bag, and the generated SQL statements are going to be more efficient.

So, let's changing the `comments` collection to use a `Set` this time:

```
@ElementCollection
private Set<String> comments = new HashSet<>();
```

After inserting the same 3 comments, as we did when using a `List` with `@ElementCollection`, let's see how the `Set` behaves when it comes to removing elements.

When removing the first element, we added to the `Set`:

```
post.getComments().remove("My first review");
```

Hibernate generates a single SQL delete statement:

```
DELETE FROM Post_comments WHERE Post_id = 1 and comments = 'My first review'
```

When removing the last element, we added to the `Set`:

```
post.getComments().remove("My third review");
```

Again, Hibernate generates a single SQL delete statement:

```
DELETE FROM Post_comments WHERE Post_id = 1 and comments = 'My third review'
```

So, if you want to use an `@ElementCollection` and benefit from its simplicity, it's much better if you are using a `Set`, instead of a `List`.

However, even if using a `Set` can render more efficient SQL statements than when using a `List`, it's still much better if you replace the `@ElementCollection` with either a bidirectional `@OneToMany` association or just a simple `@ManyToOne` association that maps the Foreign Key column.

This way, you can remove elements without having to fetch the collection via the parent entity.

10.5 @OneToOne

From a database perspective, the one-to-one association is based on a foreign key that is constrained to be unique. This way, a parent row can be referenced by at most one child record only.

In JPA, the `@OneToOne` relationship can be either unidirectional or bidirectional.

10.5.1 Unidirectional @OneToOne

In the following example, the `Post` entity represents the parent-side, while the `PostDetails` is the child-side of the one-to-one association.

As already mentioned, the JPA entity relationship diagram matches exactly the one-to-one table relationship.

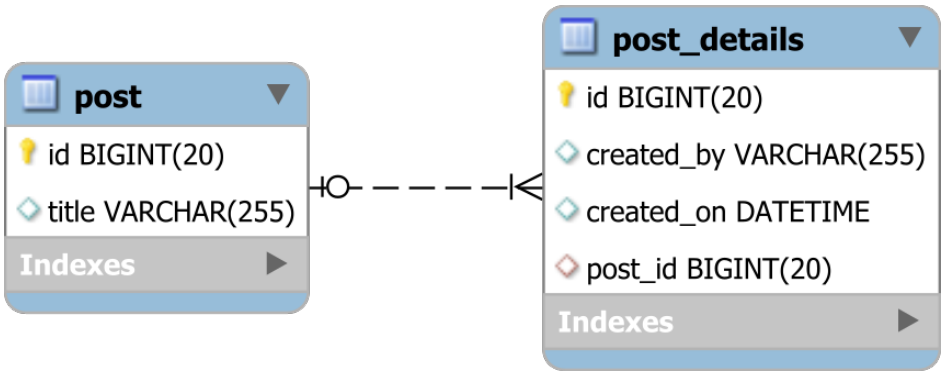

Figure 10.9: The one-to-one table relationship

Even from the Domain Model side, the unidirectional `@OneToOne` relationship is strikingly similar to the unidirectional `@ManyToOne` association.

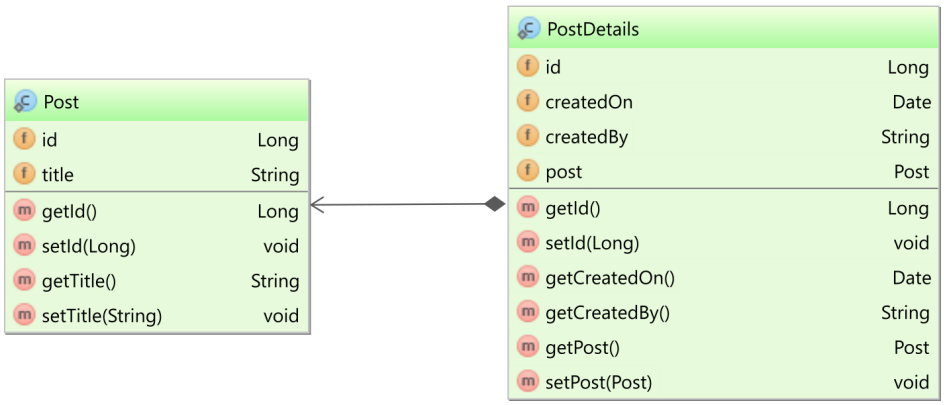

Figure 10.10: The unidirectional @OneToOne relationship

The mapping is done through the `@OneToOne` annotation, which, just like the `@ManyToOne` mapping, might also take a `@JoinColumn` as well.

```
@OneToOne
@JoinColumn(name = "post_id")
private Post post;
```

The unidirectional `@OneToOne` association controls the associated foreign key, so, when the `post` attribute is set:

```
Post post = entityManager.find(Post.class, 1L);
PostDetails details = new PostDetails("John Doe");
details.setPost(post);
entityManager.persist(details);
```

Hibernate populate the foreign key column with the associated `post` identifier:

```
INSERT INTO post_details (created_by, created_on, post_id, id)
VALUES ('John Doe', '2016-01-08 11:28:21.317', 1, 2)
```

Even if this is a unidirectional association, the `Post` entity is still the parent-side of this relationship. To fetch the associated `PostDetails`, a JPQL query is needed:

```
PostDetails details = entityManager.createQuery(
    "select pd " +
    "from PostDetails pd " +
    "where pd.post = :post", PostDetails.class)
.setParameter("post", post)
.getSingleResult();
```

If the `Post` entity always needs its `PostDetails`, a separate query might not be desirable. To overcome this limitation, it is important to know the `PostDetails` identifier prior to loading the entity.

One workaround would be to use a `@NaturalId`, which might not require a database access if the entity is stored in the second-level cache. Fortunately, there is even a simpler approach which is also portable across JPA providers as well. The JPA 2.0 specification added support for derived identifiers, making possible to link the `PostDetails` identifier to the `post` table primary key.

This way, the `post_details` table primary key can also be a foreign key referencing the `post` table identifier.